Project Plan: EventHub Management System

Group 9

I. Title Page

- **Project Title:** EventHub – A Modern RESTful Event Management Solution
- **Semester & Year:** Spring 2026
- **Team Members:**
    - **Ali Faisal** (ID: 415002117)
    - **Syed Rayan** (ID: 764000072)

II. Executive Summary

EventHub is a web application that is designed to bring event discovery and registration and participation management together all in one place. The project shifts from a traditional solid design towards a separated N-tier architecture built on Spring Boot and RESTful APIs. The system allows users to explore events and register for events through a dedicated subscription model, with administrators simultaneously maintaining and managing data integrity through the backend. The main goal to achieve a scalable and persistent platform that is capable of handling real-time event coordination.

III. Project Scope and Timeline

EventHub encompasses the full event lifecycle, from the creations made by the admins to the users subscribed.

- **Scope**: The system controlled four major entities: Events, Users, Subscriptions, and Administrators.
- **Timeline:**
    - **Phase 1:** Gathering Requirements, ERD Design, and initial UI and frontend prototyping using Mustache templates.
    - **Phase 2:** Implementing JPA Persistence, migration to MySQL for database, development of REST controllers, and Bootstrap-based frontend implemented using Fetch API.

IV. Key Features

- **Complete CRUD Capabilities:** Separate REST Controllers for each entity allow for the full creation, deletion, retrieval, deletion, and updates of records.
- **Persistent Data Storage**: Integration of a MySQL database using Spring Data JPA to ensure data remains when the application restarts.
- **Dynamic Data Filtering:** Custom search filtering possible utilizing JPQL to filter and find events by title or administrators of departments.
- **Asynchronous Interface:** A professional, responsive frontend that updates data in real-time without needing to reload the page.

V. User / Interaction Scenario

1. A user visits events.html, where current available events are automatically pulled from the /api/events endpoint.
2. When the user finds an event of interest, for example, a Tech Workshop event, they head to the subscription page where they fill out a form and enter their details, and the JavaScript fetch() API sends a POST request to the backend.
3. An administrators logs into the administrators.html dashboard to review the new subscriptions. By using a PUT request, the admin can the subscription status and access levels.

VI. Implementation Details

- The backend is built on Spring Boot 3.2.4, streamlining development and dependancy management.
- Spring Data JPA with Hibernate handles persistence, bridging Java entities and relational database tables.
- The frontend uses HTML5, CSS3, and ES6 JavaScript, with bootstrap 5 ensuring a responsive and consistent layout.
- Intelliji IDEA serves as the main development platform, with Maven managing builds and Git handling version control and branch based collaboration.

VII. Anticipated Challenges / Limitations / Mitigations

- **Challenge**: Making sure data is consistent between the User and Subscription tables during high-frequency updates.
    - **Mitigation**: Implementing the @Transactional logic in the service layer to enable automatic database operations.

- **Challenge**: Shifting from server side Mustache rendering to a client side Fetch API architecture
    - **Mitigation**: Required reconstruction. The solution was adopting a strictly decoupled approach where the backend exclusively serves JSON and the frontend independently manages all the UI logic.

- **Limitation**: The system currently runs on localhost, which restricts external access. A potential fix in future phases would be deploying the backend and database folders to a cloud platform such as AWS or Heroku.